08_agents_provenance

August 18, 2026

1 NB8 — Agents as consumers, and provenance as a legal deadline

Stack: `deltalake` + DuckDB. No LLM call, no API key — the agent loop is deterministic so the *data contract* is what you study, not model behaviour. Maps to slide §11 (Agent Memory & Trajectory, MCP 2026-07-28) + §12 (Provenance) + deliverable bullet 8.

Three parts:

1. **Trajectories** — the lakehouse as system-of-record for what an agent did
2. **MCP** — the protocol shape an agent uses to reach your catalog
3. **Provenance** — EU AI Act Art. 10, in force since **2 Aug 2026**

[1]:
```
import _setup  # noqa: F401

import json
import time
from pathlib import Path

import duckdb
import polars as pl
import pyarrow as pa
from deltalake import DeltaTable, write_deltalake

from lakehouse import catalog, namespace, path, reset, reset_catalog, to_arrow

import generate_ai_data as gen

TRACES = path("bronze", "agent_traces")
DOCS = path("bronze", "docs_multimodal")
if not Path(TRACES).exists() or not Path(DOCS).exists():
    gen.main()

con = duckdb.connect()
con.register("bronze_traces", DeltaTable(TRACES).to_pyarrow_table())
print(f"Bronze trajectory steps: {DeltaTable(TRACES).count():,}")
```

```
Bronze trajectory steps: 1,578
```

1.1 Part 1 — Trajectories through the medallion

> A **trajectory** (rollout) is a sequence of `(observation, action, reward)` from the initial state to termination — the fuel an RL update consumes.

It differs from supervised data in one way that changes your storage design: **the data distribution shifts as the policy improves**, so a static dataset is useless. Trajectory tables are therefore *append-heavy, versioned, and sliceable by policy version.*

Bronze = raw steps · Silver = parsed + `trajectory_id` · Gold = per-agent rollups.

[2]:
```
SILVER = path("silver", "agent_trajectories")
GOLD = path("gold", "agent_performance")
reset(SILVER, GOLD)

# Two policy versions in flight - the partition key the slide insists on.
silver = to_arrow(con.sql("""
    SELECT
        session_id                          AS trajectory_id,
        step,
        tool,
        status,
        reward,
        subject_id,
        input_tokens + output_tokens        AS total_tokens,
        latency_ms,
        (input_tokens * 3.0 + output_tokens * 15.0) / 1e6 AS cost_usd,
        CASE WHEN CAST(substr(session_id, 6) AS INT) < 150
             THEN 'policy-v2' ELSE 'policy-v3' END        AS agent_version
    FROM bronze_traces
"""))

# Partitioning by agent_version lets you drop or retrain on one policy's
# rollouts without touching the other's.
write_deltalake(SILVER, silver, mode="overwrite",␣
 ↪partition_by=["agent_version"])
con.register("silver", DeltaTable(SILVER).to_pyarrow_table())
print(f"Silver: {silver.num_rows:,} steps, partitioned by agent_version")
print(f"  partitions on disk: {sorted(p.name for p in Path(SILVER).
 ↪glob('agent_version=*'))}")
```

```
Silver: 1,578 steps, partitioned by agent_version
  partitions on disk: ['agent_version=policy-v2', 'agent_version=policy-v3']
```

[3]:
```
gold = to_arrow(con.sql("""
    WITH per_traj AS (
        SELECT trajectory_id, agent_version,
               max(reward)              AS success,
               count(*)                 AS steps,
```

```
            sum(cost_usd)            AS cost_usd,
            sum(latency_ms)          AS latency_ms,
            max(status = 'error')    AS had_error
        FROM silver GROUP BY 1, 2
    )
    SELECT agent_version,
           count(*)                                AS trajectories,
           round(avg(success), 3)                  AS success_rate,
           round(avg(steps), 2)                    AS avg_steps,
           round(avg(cost_usd), 5)                 AS avg_cost_usd,
           round(sum(cost_usd), 2)                 AS total_cost_usd,
           round(avg(latency_ms) / 1000, 1)        AS avg_seconds
    FROM per_traj GROUP BY 1 ORDER BY 1
"""))
write_deltalake(GOLD, gold, mode="overwrite")
print(pl.from_arrow(gold))
```

```
shape: (2, 7)
```

agent_version	trajectories	success_rate	avg_steps	avg_cost_usd	total_cost_usd	avg_seconds
str	i64	f64	f64	f64	f64	f64
policy-v2	150	0.76	5.26	0.06915	10.37	16.2
policy-v3	150	0.753	5.26	0.06928	10.39	15.7

1.1.1 The reproducibility contract: pin the table version into the run

The slide's rule: *pin the trajectory table version into the training run* — the same contract as MLflow Delta version. Without it, "which data did this policy train on?" has no answer, and Annex IV has no answer either.

[4]:
```
training_run = {
    "run_id": "rl-run-2026-08-17-001",
    "policy": "policy-v4",
    "trajectory_table": SILVER,
    "table_version": DeltaTable(SILVER).version(),     # ← the pin
```

```
    "n_steps_seen": DeltaTable(SILVER).count(),
}
print(json.dumps(training_run, indent=2, default=str))

# The world moves on: new rollouts land.
write_deltalake(SILVER, silver.slice(0, 400), mode="append",␣
 ↪partition_by=["agent_version"])
print(f"\nAfter more rollouts land - table version {DeltaTable(SILVER).
 ↪version()}, "
      f"{DeltaTable(SILVER).count():,} steps")

# Six months later, an auditor asks what the run actually saw.
pinned = DeltaTable(SILVER, version=training_run["table_version"])
print(f"Replay at pinned version {training_run['table_version']}: {pinned.
 ↪count():,} steps")
print(f"Matches what training saw: {pinned.count() ==␣
 ↪training_run['n_steps_seen']}")
print("\nThat one integer is the difference between a reproducible run and a␣
 ↪story.")
```

```
{
  "run_id": "rl-run-2026-08-17-001",
  "policy": "policy-v4",
  "trajectory_table": "D:\\AI20K\\Track2\\Day18-Track2-Lakehouse-
Lab\\_lakehouse\\silver\\agent_trajectories",
  "table_version": 0,
  "n_steps_seen": 1578
}

After more rollouts land - table version 1, 1,978 steps
Replay at pinned version 0: 1,578 steps
Matches what training saw: True

That one integer is the difference between a reproducible run and a story.
```

1.2 Part 2 — MCP: the shape of the agent ↔ lakehouse boundary

MCP revision 2026-07-28 made changes that matter specifically to a data platform. We implement the *shape* of each below — no network, no LLM.

Change	Why a data team cares
Stateless core — no `initialize` handshake; each request self-describes in `_meta`	A catalog MCP server can sit behind a round-robin load balancer with no session store
Cacheable lists — `tools/list` carries `ttlMs`, `cacheScope`	A 50,000-table catalog stops re-listing itself every agent turn
Multi-round-trip — `resultType: input_required`	Human-in-the-loop before `DELETE` or a cross-border export

Change	Why a data team cares
Header routing — `Mcp-Method`, `Mcp-Name`	Gateway routes and **meters per tool** without parsing JSON
Tasks extension — poll `tasks/get`	The right shape for a 40-minute Spark job

[5]:
```
CAT = "nb8"          # own catalog dir - see scripts/lakehouse.py:_catalog_dir
reset_catalog(CAT)
cat = catalog(CAT)
ns = namespace(cat, "lake")
ice = cat.create_table(f"{ns}.trajectories", schema=pa.schema([
    pa.field("trajectory_id", pa.string(), nullable=False),
    pa.field("agent_version", pa.string()),
    pa.field("reward", pa.float64()),
]))
ice.append(pa.table({
    "trajectory_id": to_arrow(con.sql("SELECT DISTINCT trajectory_id FROM␣
 ↪silver ORDER BY 1")).column(0),
    "agent_version": to_arrow(con.sql("SELECT DISTINCT ON (trajectory_id)␣
 ↪agent_version FROM silver ORDER BY trajectory_id")).column(0),
    "reward": to_arrow(con.sql("SELECT max(reward) FROM silver GROUP BY␣
 ↪trajectory_id ORDER BY trajectory_id")).column(0),
}, schema=ice.schema().as_arrow()))
print(f"Catalog now serves: {cat.list_tables(ns)}")
```

```
Catalog now serves: [('lake', 'trajectories')]
```

[6]:
```
class LakehouseMCP:
    """An MCP-2026-07-28-shaped read surface over the catalog.

    Deliberately NOT a network server - the point is the contract, not the
    transport. Every behaviour below maps to a row of the table above.
    """

    DESTRUCTIVE = {"drop_table", "delete_rows"}

    def __init__(self, cat, ns: str, list_ttl_ms: int = 60_000):
        self.cat, self.ns = cat, ns
        self.list_ttl_ms = list_ttl_ms
        self._cache: dict[str, tuple[float, object]] = {}
        self.meter: dict[str, dict] = {}          # per-tool billing, keyed by␣
 ↪Mcp-Name
        self.catalog_reads = 0                    # how often we really hit the␣
 ↪catalog

    #   Cacheable lists: tools/list advertises its own TTL and scope
    def tools_list(self) -> dict:
```

```
        return {
            "tools": [
                {"name": "list_tables",  "description": "Enumerate tables in a␣
↪namespace"},
                {"name": "get_schema",   "description": "Column names and␣
↪types"},
                {"name": "query",        "description": "Read-only SQL over a␣
↪table"},
                {"name": "submit_scan",  "description": "Long scan; returns a␣
↪task handle"},
                {"name": "delete_rows",  "description": "DESTRUCTIVE: delete␣
↪matching rows"},
            ],
            # The client may cache this list for ttlMs, scoped per-session.
            "_meta": {"ttlMs": self.list_ttl_ms, "cacheScope": "session"},
        }

    #   Stateless core: no handshake, no Mcp-Session-Id; _meta per request
    def call(self, name: str, args: dict | None = None, _meta: dict | None =␣
↪None) -> dict:
        args, _meta = args or {}, _meta or {}
        t0 = time.perf_counter()

        # Header-style routing + metering. A gateway can do this without
        # parsing the JSON body at all.
        headers = {"Mcp-Method": "tools/call", "Mcp-Name": name}

        # Multi-round-trip: destructive tools stop and ask a human first.
        if name in self.DESTRUCTIVE and not _meta.get("confirmed"):
            return {"resultType": "input_required",
                    "prompt": f"Confirm {name}({args}) - this cannot be undone␣
↪by the agent.",
                    "_meta": {"headers": headers, "requiresConfirmation": True}}

        result = self._dispatch(name, args)
        elapsed_ms = (time.perf_counter() - t0) * 1000
        m = self.meter.setdefault(name, {"calls": 0, "ms": 0.0})
        m["calls"] += 1
        m["ms"] += elapsed_ms
        return {"resultType": "ok", "result": result,
                "_meta": {"headers": headers, "elapsedMs": round(elapsed_ms,␣
↪2)}}

    def _dispatch(self, name: str, args: dict):
        if name == "list_tables":
            key = f"list:{self.ns}"
```

```
            hit = self._cache.get(key)
            if hit and (time.time() - hit[0]) * 1000 < self.list_ttl_ms:
                return {"tables": hit[1], "cached": True}
            self.catalog_reads += 1
            tables = [f"{a}.{b}" for a, b in self.cat.list_tables(self.ns)]
            self._cache[key] = (time.time(), tables)
            return {"tables": tables, "cached": False}

        if name == "get_schema":
            t = self.cat.load_table(args["table"])
            return {"columns": [{"name": f.name, "type": str(f.field_type),
                                 "required": f.required} for f in t.schema().
 ↪fields]}

        if name == "query":
            t = self.cat.load_table(args["table"])
            # Guardrail: the agent never gets an unbounded scan.
            limit = min(int(args.get("limit", 10)), 100)
            return {"rows": t.scan(limit=limit).to_arrow().to_pylist()}

        if name == "submit_scan":
            # Tasks extension: return a handle immediately, poll for completion.
            # Same shape as Iceberg 1.11 server-side planning returning a␣
 ↪plan-id.
            task_id = f"task_{len(self._cache):04d}"
            self._cache[task_id] = (time.time(), args["table"])
            return {"taskId": task_id, "status": "working", "pollAfterMs": 50}

        if name == "delete_rows":
            return {"deleted": 0, "note": "wired to a no-op in the lab"}

        raise ValueError(f"unknown tool: {name}")

    def tasks_get(self, task_id: str) -> dict:
        created, table = self._cache[task_id]
        if (time.time() - created) * 1000 < 50:
            return {"status": "working"}
        n = self.cat.load_table(table).scan().to_arrow().num_rows
        return {"status": "completed", "result": {"rows": n}}


mcp = LakehouseMCP(cat, ns)
```

1.2.1 Cacheable lists: a 50,000-table catalog should not re-list every turn

[7]:
```
for turn in range(5):
    r = mcp.call("list_tables")
    print(f"  turn {turn}: cached={r['result']['cached']} ␣
 ↪tables={r['result']['tables']}")
print(f"\nActual catalog round-trips for 5 agent turns: {mcp.catalog_reads}")
print(f"tools/list advertises: {mcp.tools_list()['_meta']}")
```

```
  turn 0: cached=False  tables=['lake.trajectories']
  turn 1: cached=True  tables=['lake.trajectories']
  turn 2: cached=True  tables=['lake.trajectories']
  turn 3: cached=True  tables=['lake.trajectories']
  turn 4: cached=True  tables=['lake.trajectories']

Actual catalog round-trips for 5 agent turns: 1
tools/list advertises: {'ttlMs': 60000, 'cacheScope': 'session'}
```

1.2.2 Human-in-the-loop before anything destructive

[8]:
```
attempt = mcp.call("delete_rows", {"table": f"{ns}.trajectories", "where":␣
 ↪"reward = 0"})
print(f"resultType: {attempt['resultType']}")
print(f"prompt:     {attempt['prompt']}")

approved = mcp.call("delete_rows", {"table": f"{ns}.trajectories", "where":␣
 ↪"reward = 0"},
                    _meta={"confirmed": True})
print(f"\nafter human approval → resultType: {approved['resultType']}")
print("\nThe agent CANNOT self-approve. That gate is the protocol's, not the␣
 ↪model's.")
```

```
resultType: input_required
prompt:     Confirm delete_rows({'table': 'lake.trajectories', 'where': 'reward
= 0'}) - this cannot be undone by the agent.

after human approval → resultType: ok

The agent CANNOT self-approve. That gate is the protocol's, not the model's.
```

1.2.3 Tasks: the right shape for a scan that takes 40 minutes

[9]:
```
sub = mcp.call("submit_scan", {"table": f"{ns}.trajectories"})
task_id = sub["result"]["taskId"]
print(f"submit_scan → {sub['result']}")
for poll in range(5):
    st = mcp.tasks_get(task_id)
    print(f"  tasks/get #{poll}: {st['status']}")
```

```
    if st["status"] == "completed":
        print(f"  result: {st['result']}")
        break
    time.sleep(0.03)
print("\nIceberg 1.11 server-side planning returns a plan-id you poll the same␣
 ↪way.")
print("Two protocols, one shape - that is not a coincidence.")
```

```
submit_scan → {'taskId': 'task_0001', 'status': 'working', 'pollAfterMs': 50}
  tasks/get #0: working
  tasks/get #1: working
  tasks/get #2: completed
  result: {'rows': 300}

Iceberg 1.11 server-side planning returns a plan-id you poll the same way.
Two protocols, one shape - that is not a coincidence.
```

1.2.4 Per-tool metering — the FinOps hook

Because `Mcp-Name` is a header, a gateway can bill per tool without ever parsing the request body.

[10]:
```
mcp.call("get_schema", {"table": f"{ns}.trajectories"})
mcp.call("query", {"table": f"{ns}.trajectories", "limit": 3})
print(f"{'tool':<14} {'calls':>6} {'total ms':>10}")
for tool, m in sorted(mcp.meter.items()):
    print(f"{tool:<14} {m['calls']:>6} {m['ms']:>10.2f}")
```

```
tool            calls   total ms
delete_rows         1       0.00
get_schema          1       2.55
list_tables         5       1.76
query               1       5.78
submit_scan         1       0.01
```

1.3 Part 3 — Provenance: EU AI Act Art. 10 is already in force

High-risk obligations applied from **2 August 2026** — that date has passed. Article 10 attaches to your *training / validation / test* sets: origin, how they were prepared (labelling, cleaning), bias checks, and data gaps.

The slide's key reframe:

> Every training row must resolve to exactly one of four buckets — **licensed**, **public domain**, **scraped with opt-out checked**, **synthetic (with generator recorded)**. Those four buckets are **one governed column plus a partition key**, not a Confluence page.

[11]:
```
docs = DeltaTable(DOCS).to_pyarrow_table()
con.register("docs", docs)
```

```
# The four Art. 10 buckets, as ONE column expression. Anything that falls
# through is UNCLASSIFIED - and UNCLASSIFIED is an audit finding, not a
# rounding error, so it must never silently become a default bucket.
BUCKET_SQL = """
    CASE
        WHEN license IN ('proprietary', 'commercial')          THEN 'licensed'
        WHEN license = 'cc-by-4.0'                             THEN␣
 ↪'public_domain'
        WHEN license = 'user-owned'  AND consent_train         THEN␣
 ↪'scraped_optout_checked'
        WHEN license = 'synthetic'   AND generator IS NOT NULL THEN 'synthetic'
        ELSE 'UNCLASSIFIED'
    END
"""
audit = to_arrow(con.sql(f"""
    SELECT {BUCKET_SQL} AS provenance_bucket,
           count(*) AS rows,
           round(100.0 * count(*) / sum(count(*)) OVER (), 1) AS pct
    FROM docs GROUP BY 1 ORDER BY rows DESC
"""))
print(pl.from_arrow(audit))

unclassified = con.sql(f"SELECT count(*) FROM docs WHERE {BUCKET_SQL} =␣
 ↪'UNCLASSIFIED'").fetchone()[0]
print(f"\nUNCLASSIFIED rows: {unclassified:,}")
print("→ Mixing scraped and licensed data in one unlabelled bucket is a 2026␣
 ↪audit failure.")
```

```
shape: (5, 3)

 provenance_bucket        rows   pct
 ---                      ---    ---
 str                      i64    f64

 licensed                 675    33.8
 UNCLASSIFIED             334    16.7
 public_domain            333    16.7
 synthetic                331    16.6
 scraped_optout_checked   327    16.4


UNCLASSIFIED rows: 334
→ Mixing scraped and licensed data in one unlabelled bucket is a 2026 audit
failure.
```

1.3.1 Make the bucket a real column *and* a partition key

Once provenance is a partition, "exclude everything we cannot defend" is a partition prune, not a full-table scan and a prayer.

[12]:
```
GOVERNED = path("silver", "training_corpus_governed")
reset(GOVERNED)
governed = to_arrow(con.sql(f"""
    SELECT doc_id, title, topic, subject_id, source, license, consent_train,␣
 ↪generator,
           {BUCKET_SQL} AS provenance_bucket
    FROM docs
"""))
write_deltalake(GOVERNED, governed, mode="overwrite",␣
 ↪partition_by=["provenance_bucket"])

parts = sorted(p.name for p in Path(GOVERNED).glob("provenance_bucket=*"))
print("Partitions on disk:")
for p_ in parts:
    print(f"  {p_}")

con.register("governed", DeltaTable(GOVERNED).to_pyarrow_table())
trainable = con.sql("""SELECT count(*) FROM governed
                       WHERE provenance_bucket <> 'UNCLASSIFIED'""").
 ↪fetchone()[0]
print(f"\nDefensible training rows: {trainable:,} / {governed.num_rows:,}")
print(f"Excluded as UNCLASSIFIED:  {governed.num_rows - trainable:,}")
```

```
Partitions on disk:
  provenance_bucket=UNCLASSIFIED
  provenance_bucket=licensed
  provenance_bucket=public_domain
  provenance_bucket=scraped_optout_checked
  provenance_bucket=synthetic


Defensible training rows: 1,666 / 2,000
Excluded as UNCLASSIFIED:  334
```

1.3.2 The Annex IV answer: "which corpus version was model X trained on?"

`DESCRIBE HISTORY` + the pinned version + the run id. Three facts, one query.

[13]:
```
corpus_version = DeltaTable(GOVERNED).version()
model_card = {
    "model": "vinuni-rag-v1",
    "corpus_table": "silver.training_corpus_governed",
    "corpus_version": corpus_version,
    "rows_used": trainable,
```

```
    "buckets_used": [p_.split("=", 1)[1] for p_ in parts if "UNCLASSIFIED" not␣
 ↪in p_],
    "excluded_rows": governed.num_rows - trainable,
    "exclusion_reason": "license=unknown → fails Art. 10 origin requirement",
}
print(json.dumps(model_card, indent=2))

hist = DeltaTable(GOVERNED).history()
print(f"\nDESCRIBE HISTORY → {len(hist)} version(s); v{corpus_version} written "
      f"by {hist[0]['operation']}")
```

```
{
  "model": "vinuni-rag-v1",
  "corpus_table": "silver.training_corpus_governed",
  "corpus_version": 0,
  "rows_used": 1666,
  "buckets_used": [
    "licensed",
    "public_domain",
    "scraped_optout_checked",
    "synthetic"
  ],
  "excluded_rows": 334,
  "exclusion_reason": "license=unknown \u2192 fails Art. 10 origin requirement"
}

DESCRIBE HISTORY → 1 version(s); v0 written by WRITE
```

1.3.3 Right-to-erasure, and why provenance makes it answerable

Vietnam's **PDPL (Law 91/2025)** and the GDPR both give a data subject the right to erasure. The question that sinks teams is not *"can you delete it?"* — it is *"can you prove what it was in, including the model corpus?"*

[14]:
```
SUBJECT = "user_007"
before = con.sql(f"SELECT count(*) FROM governed WHERE subject_id =␣
 ↪'{SUBJECT}'").fetchone()[0]

# What was this subject's data used for? Provenance answers it directly.
usage = to_arrow(con.sql(f"""
    SELECT provenance_bucket, count(*) AS rows
    FROM governed WHERE subject_id = '{SUBJECT}' GROUP BY 1
""")).to_pylist()
print(f"Erasure request from {SUBJECT}: {before} rows, used as: {usage}")

dt = DeltaTable(GOVERNED)
dt.delete(f"subject_id = '{SUBJECT}'")
after_dt = DeltaTable(GOVERNED)
```

```
con.register("governed_after", after_dt.to_pyarrow_table())
after = con.sql(f"SELECT count(*) FROM governed_after WHERE subject_id =␣
 ↪'{SUBJECT}'").fetchone()[0]

print(f"\nRows for {SUBJECT}: {before} → {after}")
print(f"Table version: {corpus_version} → {after_dt.version()}")
print(f"""
Note the tension the slide flags: time travel means v{corpus_version} STILL␣
 ↪contains
the erased rows. Deletion is only complete once retention expires those
versions (NB6, Job 3). "We support time travel" and "we honour erasure" are
in direct conflict unless your retention window is a deliberate, written
decision - not a default.""")
```

```
Erasure request from user_007: 8 rows, used as: [{'provenance_bucket':
'scraped_optout_checked', 'rows': 1}, {'provenance_bucket': 'licensed', 'rows':
1}, {'provenance_bucket': 'UNCLASSIFIED', 'rows': 5}, {'provenance_bucket':
'synthetic', 'rows': 1}]


Rows for user_007: 8 → 0
Table version: 0 → 1

Note the tension the slide flags: time travel means v0 STILL contains
the erased rows. Deletion is only complete once retention expires those
versions (NB6, Job 3). "We support time travel" and "we honour erasure" are
in direct conflict unless your retention window is a deliberate, written
decision - not a default.
```

1.4 NB8 pass criteria

Check	Target
Trajectory medallion	Silver partitioned by `agent_version`; Gold has both policies
Version pin	Replay at the pinned version matches what training saw
MCP cacheable lists	5 agent turns → 1 catalog round-trip
MCP human-in-the-loop	Destructive call returns `input_required` before approval
MCP tasks	`submit_scan` → poll → `completed`
Provenance	all 4 Art. 10 buckets exist as partitions; UNCLASSIFIED excluded
Erasure	Subject rows = 0 in the current version, and you can say which bucket they were in

[15]:
```
checks = {
```

```
    "silver partitioned by agent_version": len(list(Path(SILVER).
 ↪glob("agent_version=*"))) == 2,
    "gold covers both policies":           gold.num_rows == 2,
    "version pin replays exactly":         pinned.count() ==␣
 ↪training_run["n_steps_seen"],
    "5 turns → 1 catalog read":            mcp.catalog_reads == 1,
    "destructive needs confirmation":      attempt["resultType"] ==␣
 ↪"input_required",
    "confirmed call proceeds":             approved["resultType"] == "ok",
    "tasks poll completes":                st["status"] == "completed",
    "all 4 Art.10 buckets present":        len([x for x in parts if␣
 ↪"UNCLASSIFIED" not in x]) == 4,
    "unclassified rows found":             unclassified > 0,
    "erasure removed subject rows":        after == 0 and before > 0,
}
for k, v in checks.items():
    print(f"  [{'PASS' if v else 'FAIL'}] {k}")
assert all(checks.values()), "NB8 incomplete - see FAIL rows above"
print("\nNB8 complete.")
```

```
  [PASS] silver partitioned by agent_version
  [PASS] gold covers both policies
  [PASS] version pin replays exactly
  [PASS] 5 turns → 1 catalog read
  [PASS] destructive needs confirmation
  [PASS] confirmed call proceeds
  [PASS] tasks poll completes
  [PASS] all 4 Art.10 buckets present
  [PASS] unclassified rows found
  [PASS] erasure removed subject rows

NB8 complete.
```